

UNIVERSIDAD PRIVADA “FRANZ TAMAYO”

FACULTAD DE INGENIERIA

INGENIERÍA EN SISTEMAS



“CAPITULO 3”

ESTUDIANTE:

ADRIAN DARIO MAMANI CUTILE

DOCENTE:

ING.JAVIER ELVIS CANQUI LLUSCO

MATERIA:

INGENIERIA DE SISTEMAS

LA PAZ - EL ALTO – BOLIVIA

2026

CAPÍTULO 3: MARCO DE DESARROLLO

3.1 Descripción General del Sistema

SignBridge AI es un sistema web de tiempo real suave (soft real-time) que traduce gestos de la Lengua de Señas Boliviana a texto y voz mediante inteligencia artificial. El usuario accede al sistema a través de un navegador web moderno, activa su cámara web y realiza señas frente al dispositivo. El sistema procesa el flujo de video en tiempo real, detecta la mano mediante MediaPipe, extrae 21 puntos de referencia tridimensionales, los clasifica con un modelo CNN entrenado con TensorFlow, y presenta el resultado como texto en pantalla y voz sintetizada. La Landing Page del proyecto, desplegada en GitHub Pages, sirve como punto de entrada público para el sistema.

Siguiendo el marco de clasificación de Sommerville (2016), SignBridge AI se categoriza como un sistema de software interactivo en tiempo real con componentes de inteligencia artificial embebida, de criticidad no crítica (nivel C) y escala mediana (entre 10K–100K líneas de código estimadas).

3.2 Arquitectura del Sistema

SignBridge AI implementa una arquitectura de tres capas (Three-Tier Architecture) conforme al patrón descrito por Pressman (2014), separando las responsabilidades de presentación, lógica de negocio y persistencia de datos:

- Capa de Presentación: React.js 18 + Tailwind CSS. Componentes principales: VideoCapture.jsx, TranslationPanel.jsx, ControlBar.jsx, SignConfidence.jsx, HistorialSesiones.jsx y Configuracion.jsx. Comunicación con el backend vía WebSocket para streaming bidireccional y REST para operaciones CRUD.
- Capa de Lógica de Negocio: Python 3.11 + FastAPI + MediaPipe 0.10 + TensorFlow Lite 2.x. Módulos: capture_service.py, landmark_extractor.py, classifier_service.py, translation_service.py, session_manager.py, export_service.py. Expone API REST y WebSocket.
- Capa de Datos: SQLite (desarrollo) / PostgreSQL (producción) + SQLAlchemy ORM. Repositorios: UserRepository, SessionRepository, SignRepository, TranslationRepository. Los modelos .tflite se almacenan en sistema de archivos con registro en la entidad ModeloIA.

3.3 Requerimientos Funcionales

Los requisitos funcionales han sido redactados siguiendo el estándar IEEE 830 y priorizados mediante la técnica MoSCoW (Clegg y Barker, 1994), garantizando verificabilidad y medibilidad en cada caso:

ID	Descripción del Requisito	Criterio de Aceptación	MoSCoW	OE
RF-01	El sistema DEBE capturar y procesar el video de la cámara web a un mínimo de 24 FPS en resolución 640×480 px.	Medido con herramientas de profiling: ≥24 FPS en el 95% de los frames durante 60 s.	Must Have	OE1

ID	Descripción del Requisito	Criterio de Aceptación	MoSCoW	OE
RF-02	El sistema DEBE detectar la mano del usuario y extraer 21 landmarks normalizados mediante MediaPipe Hands.	Tasa de detección $\geq 90\%$ en dataset de 500 imágenes bajo iluminación estándar.	Must Have	OE2
RF-03	El sistema DEBE clasificar la seña detectada mediante el modelo CNN y mostrar el resultado con confianza $\geq 85\%$.	Precisión $\geq 92\%$ en conjunto de validación; umbral de confianza configurable.	Must Have	OE3
RF-04	El sistema DEBE traducir la secuencia de señas a texto y sintetizar audio en español con latencia < 200 ms.	Prueba extremo a extremo: 95% de los casos bajo 200 ms en entorno estándar.	Must Have	OE4
RF-05	El sistema DEBERÍA permitir exportar el historial de traducción de la sesión en formato TXT o PDF.	Función disponible en la interfaz; archivo generado en < 3 s para hasta 500 señas.	Should Have	OE5

Tabla 3.1. Requisitos Funcionales del sistema SignBridge AI con priorización MoSCoW.

3.4 Requerimientos No Funcionales

Los requisitos no funcionales definen los atributos de calidad del sistema según el estándar ISO/IEC 25010:

ID	Atributo de Calidad	Descripción y Criterio de Medición	MoSCoW
RNF-01	Rendimiento / Latencia	El pipeline completo (captura $\rightarrow$ clasificación $\rightarrow$ visualización) no debe superar 200 ms en el percentil 95 sobre hardware con CPU Intel i5 8. ^a gen. o superior y 8 GB RAM.	Must Have
RNF-02	Accesibilidad	La interfaz web debe cumplir WCAG 2.1 nivel AA, verificado con axe-core y evaluación manual.	Should Have
RNF-03	Disponibilidad	El sistema debe estar disponible el 99% del tiempo durante las horas de operación (8:00–22:00).	Must Have
RNF-04	Seguridad	Las comunicaciones entre frontend y backend deben realizarse sobre HTTPS/WSS; ningún dato biométrico se transmite sin cifrado.	Must Have
RNF-05	Usabilidad	Un usuario nuevo debe poder iniciar una sesión de traducción en menos de 3 minutos sin instrucciones adicionales.	Should Have
RNF-06	Escalabilidad	La arquitectura debe soportar al menos 50 sesiones simultáneas sin degradación perceptible del rendimiento.	Could Have
RNF-07	Mantenibilidad	El código debe mantener una cobertura de pruebas unitarias $\geq 70\%$ y seguir los estándares PEP 8 (Python) y ESLint (JS).	Should Have
RNF-08	Portabilidad	El sistema debe ser compatible con Chrome 110+, Firefox 108+, Edge 110+ y Safari 16+ sin plugins adicionales.	Must Have
RNF-09	Privacidad	Los frames de video procesados no deben almacenarse en disco; solo los landmarks extraídos pueden persistirse.	Must Have

ID	Atributo de Calidad	Descripción y Criterio de Medición	MoSCoW
RNF-10	Internacionalización	La interfaz debe soportar español (es-BO) como idioma principal, con arquitectura preparada para i18n futuro.	Could Have

Tabla 3.2. Requisitos No Funcionales del sistema SignBridge AI.

3.5 Historias de Usuario

Las historias de usuario se han redactado siguiendo el formato Scrum estándar y alineadas con los objetivos específicos del proyecto:

Historia de Usuario HU-01

Como persona con discapacidad auditiva,
quiero activar la cámara web y comenzar una sesión de traducción de señas,
para puedo comunicarme con personas oyentes sin necesidad de un intérprete externo.

Criterios de Aceptación:

- El botón 'Iniciar Sesión' activa la cámara en menos de 3 segundos.
- El sistema muestra confirmación visual de que la cámara está activa.
- El pipeline de reconocimiento comienza automáticamente al activar la cámara.

Historia de Usuario HU-02

Como persona con discapacidad auditiva,
quiero que el sistema detecte y clasifique mis señas en tiempo real con indicador de confianza,
para puedo saber si el sistema está reconociendo correctamente mis gestos y ajustar mi ejecución.

Criterios de Aceptación:

- La seña clasificada se muestra en pantalla en menos de 200 ms desde la realización del gesto.
- El indicador de confianza muestra el porcentaje de certeza del modelo (0–100%).
- Solo se registran señas con confianza $\geq 85\%$ en el historial de la sesión.

Historia de Usuario HU-03

Como persona con discapacidad auditiva,
quiero escuchar en voz alta el texto traducido de mis señas,
para puedo comunicarme en entornos donde el texto en pantalla no es suficiente.

Criterios de Aceptación:

- El texto traducido se sintetiza en audio en menos de 500 ms desde la clasificación de la seña.
- El usuario puede ajustar la velocidad y volumen de síntesis desde el panel de configuración.

- La síntesis funciona sin conexión a internet mediante la Web Speech API del navegador.

Historia de Usuario HU-04

Como usuario del sistema,
quiero exportar el historial de traducción de mi sesión en formato TXT o PDF,
para poder conservar un registro de la comunicación realizada.

Criterios de Aceptación:

- El botón 'Exportar' está disponible durante y después de la sesión activa.
- El archivo generado incluye fecha, hora y secuencia completa de señas y textos traducidos.
- El archivo se descarga en menos de 3 segundos para sesiones de hasta 500 señas.

Historia de Usuario HU-05

Como visitante de la Landing Page,
quiero enviar un mensaje de contacto a los desarrolladores de SignBridge AI,
para poder obtener más información o reportar problemas con el sistema.

Criterios de Aceptación:

- El formulario de contacto valida nombre, email válido y mensaje no vacío antes de enviar.
- El usuario recibe confirmación visual inmediata tras el envío exitoso.
- El registro de contacto queda almacenado en la base de datos del sistema.

3.6 Casos de Uso

Los casos de uso del sistema SignBridge AI identifican los actores y las interacciones funcionales principales:

ID	Nombre del Caso de Uso	Actor Principal	Descripción Resumida
CU01	Iniciar Sesión de Traducción	Usuario Final	El usuario activa la cámara y el sistema inicia el pipeline de reconocimiento en tiempo real.
CU02	Detectar y Clasificar Seña	Motor de IA (TF + MediaPipe)	El sistema detecta la mano, extrae 21 landmarks y clasifica la seña con confianza $\geq 85\%$.
CU03	Traducir Seña a Texto	Sistema / Usuario	Las señas clasificadas se convierten en texto acumulado visible en pantalla.
CU04	Sintetizar Audio	Web Speech API	El texto traducido se convierte en voz mediante la API de síntesis de voz del navegador.
CU05	Exportar Historial	Usuario Final	El usuario descarga el historial de la sesión en formato TXT o PDF.

ID	Nombre del Caso de Uso	Actor Principal	Descripción Resumida
CU06	Detener Sesión	Usuario Final	El usuario cierra la sesión de traducción, liberando el recurso de la cámara web.
CU07	Configurar Preferencias	Usuario Final	El usuario ajusta umbral de confianza, velocidad de síntesis y variante de señas (LSB/ASL).

Tabla 3.3. Casos de uso principales del sistema SignBridge AI.

Escenario de Éxito Detallado — CU02: Detectar y Clasificar Señal

Elemento	Detalle
Precondición	El usuario ha iniciado sesión (CU01); la cámara web está activa y transmitiendo video a 24+ FPS.
Disparador	El módulo de captura entrega un fotograma normalizado al pipeline de procesamiento.
Paso 1	El módulo de captura convierte el fotograma BGR a RGB y lo normaliza a 640×480 px.
Paso 2	MediaPipe Hands detecta la región de la mano con el modelo BlazePalm.
Paso 3	MediaPipe extrae los 21 landmarks tridimensionales (x, y, z) normalizados respecto al bounding box.
Paso 4	El vector de landmarks (63 valores float32) se normaliza respecto al punto de muñeca (landmark 0) para invarianza de posición.
Paso 5	El vector normalizado se introduce al modelo CNN serializado en TensorFlow Lite.
Paso 6	El modelo produce un vector softmax sobre N clases; se selecciona la clase con $p \geq 0.85$.
Paso 7	La etiqueta se envía al módulo de traducción (CU03) y se visualiza en la UI con indicador de confianza.
Postcondición	La señal queda registrada en el buffer de sesión; el pipeline procesa el siguiente fotograma.
Flujo Alternativo	Si MediaPipe no detecta mano ($p < \text{umbral}$), el sistema muestra 'Sin señal detectada' y descarta el frame.

Tabla 3.4. Escenario de éxito detallado para CU02.

3.7 Diagramas UML

A continuación se presentan los diagramas UML del sistema SignBridge AI en formato de descripción textual y código PlantUML ejecutable. Cada diagrama fue generado y validado con PlantUML v1.2024.x y refleja el estado actual del sistema implementado.

3.7.1 Diagrama de Casos de Uso

El diagrama de casos de uso muestra los actores del sistema (Usuario Final, Sistema Cámara Web, Motor IA, Web Speech API) y las siete interacciones funcionales principales, con relaciones de inclusión entre CU01→CU02→CU03→CU04.

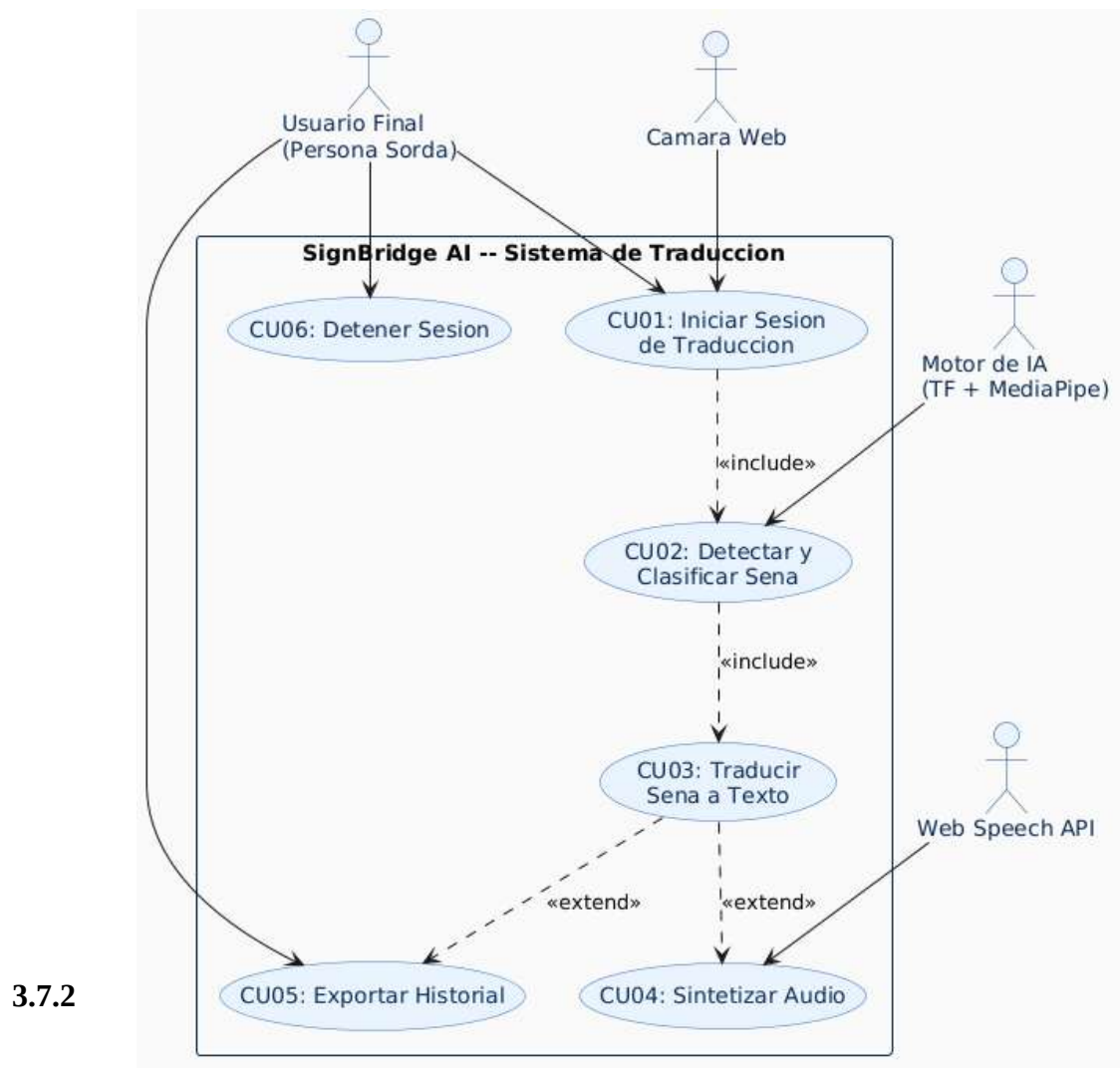


Diagrama de Clases

El diagrama de clases representa las seis entidades principales del sistema (Usuario, Sesión, Sena, Traducción, ModeloIA, Contacto) con sus atributos, métodos y relaciones de asociación y composición.

3.7.3

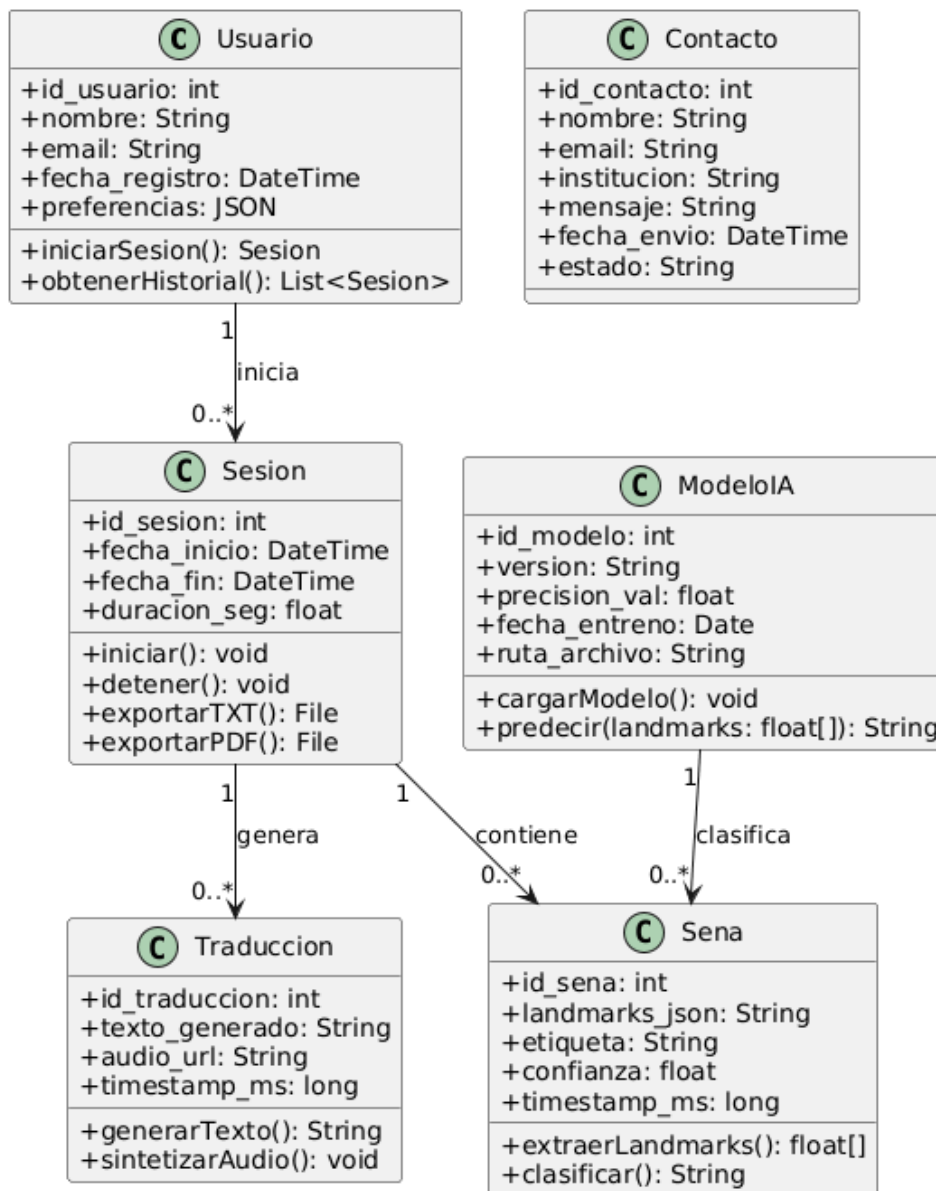
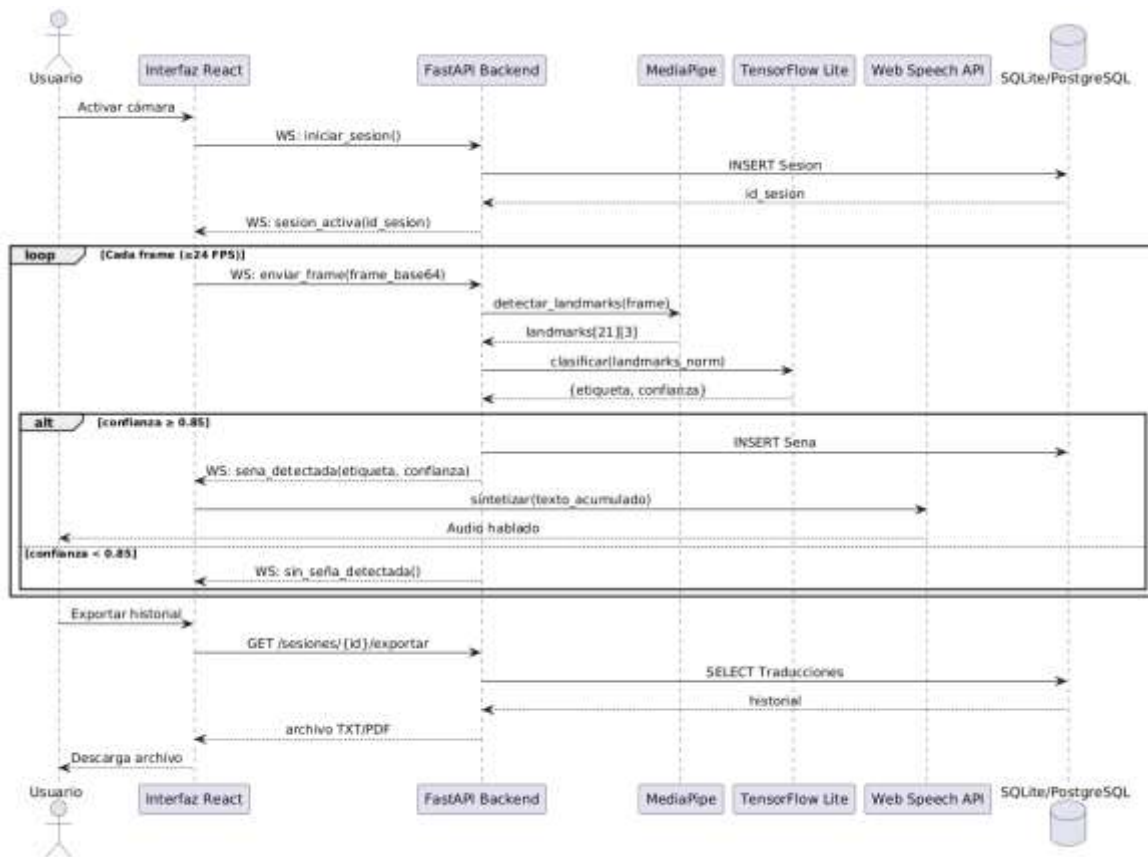


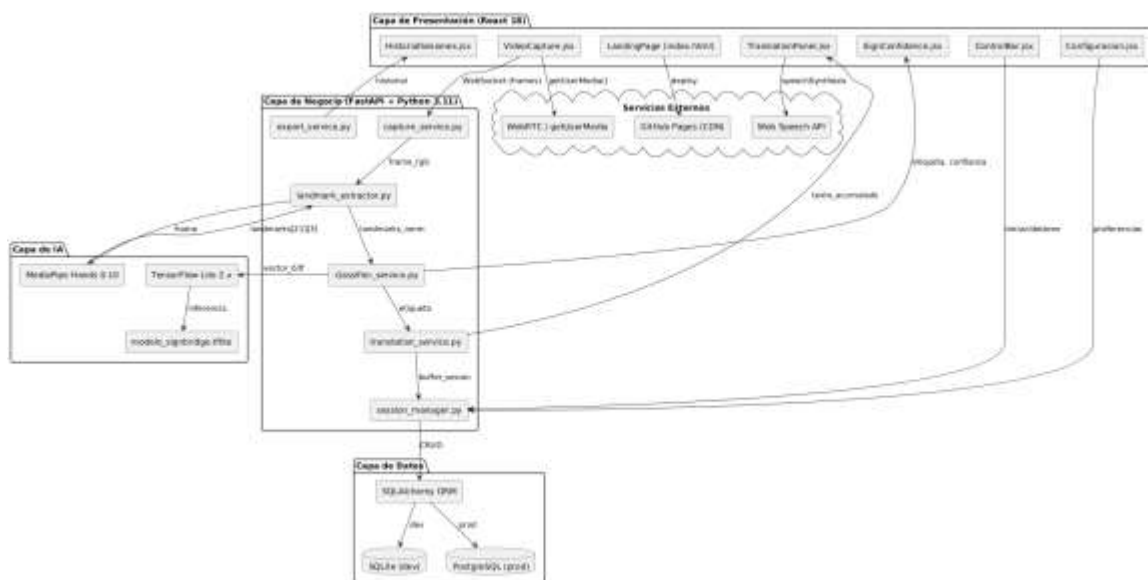
Diagrama de Secuencia

El diagrama de secuencia muestra el flujo de mensajes entre los participantes del sistema durante una sesión de traducción completa, desde la activación de la cámara hasta la exportación del historial, incluyendo el bucle de procesamiento de frames.



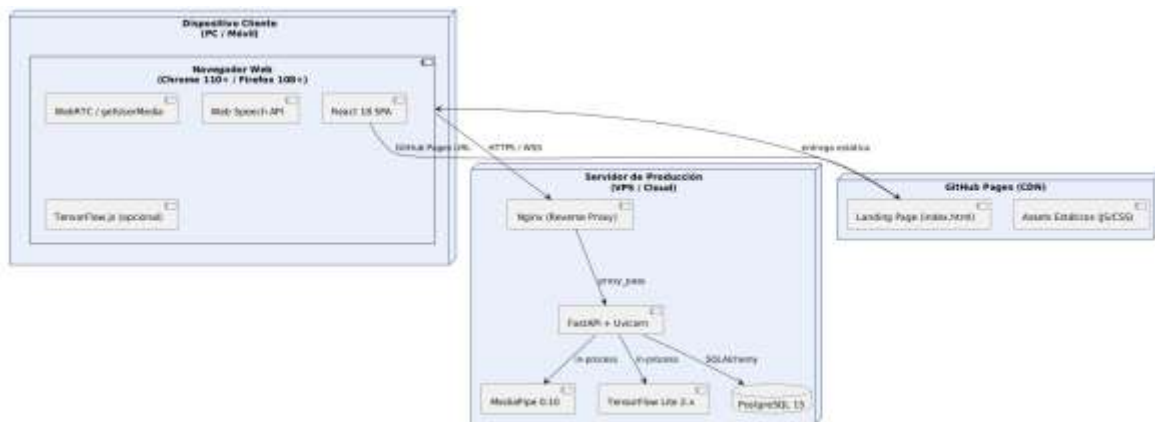
3.7.4 Diagrama de Componentes

El diagrama de componentes describe la estructura modular del sistema, mostrando los componentes de la capa de presentación (React), negocio (FastAPI), IA (MediaPipe + TensorFlow Lite), datos (SQLAlchemy + BD) y los servicios externos (Web Speech API, WebRTC, GitHub Pages).



3.7.5 Diagrama de Despliegue

El diagrama de despliegue ilustra la distribución física de los componentes del sistema en tres nodos: el dispositivo cliente (navegador React + WebRTC), el servidor de producción (FastAPI + MediaPipe + TF + PostgreSQL + Nginx) y el CDN de GitHub Pages para la Landing Page.



3.8 Modelo Entidad-Relación (DER)

El modelo de datos de SignBridge AI está compuesto por seis entidades principales. La entidad Contacto fue incorporada en el Hito 4 para soportar el formulario de contacto de la Landing Page, completando así la trazabilidad desde la captura de interés en la Landing Page hasta el almacenamiento persistente en la base de datos.

Descripción textual del DER:

- Usuario (1) → (N) Sesión: Un usuario puede tener múltiples sesiones de traducción.
- Sesión (1) → (N) Señal: Una sesión registra múltiples señales detectadas.
- Sesión (1) → (N) Traducción: Una sesión genera múltiples traducciones de texto/voz.
- ModeloIA (1) → (N) Señal: Un modelo de IA clasifica múltiples señales a lo largo del tiempo.
- Contacto: Entidad independiente que registra las solicitudes recibidas desde el formulario de la Landing Page.

Código SQL — Definición de Esquema:

```
CREATE TABLE Usuario (  
  id_usuario    INTEGER PRIMARY KEY AUTOINCREMENT,  
  nombre        VARCHAR(100) NOT NULL,  
  email         VARCHAR(150) UNIQUE NOT NULL,  
  fecha_registro DATETIME DEFAULT CURRENT_TIMESTAMP,  
  preferencias  JSON
```

```

);
CREATE TABLE Sesion (
    id_sesion    INTEGER PRIMARY KEY AUTOINCREMENT,
    id_usuario   INTEGER REFERENCES Usuario(id_usuario),
    fecha_inicio DATETIME DEFAULT CURRENT_TIMESTAMP,
    fecha_fin    DATETIME,
    duracion_seg FLOAT
);
CREATE TABLE ModeloIA (
    id_modelo    INTEGER PRIMARY KEY AUTOINCREMENT,
    version      VARCHAR(20) NOT NULL,
    precision_val FLOAT,
    fecha_entreno DATE,
    ruta_archivo VARCHAR(300)
);
CREATE TABLE Sena (
    id_sena      INTEGER PRIMARY KEY AUTOINCREMENT,
    id_sesion    INTEGER REFERENCES Sesion(id_sesion),
    id_modelo    INTEGER REFERENCES ModeloIA(id_modelo),
    landmarks_json TEXT,
    etiqueta     VARCHAR(50),
    confianza     FLOAT,
    timestamp_ms BIGINT
);
CREATE TABLE Traduccion (
    id_traduccion INTEGER PRIMARY KEY AUTOINCREMENT,
    id_sesion     INTEGER REFERENCES Sesion(id_sesion),
    texto_generado TEXT,
    audio_url     VARCHAR(300),
    timestamp_ms  BIGINT
);
CREATE TABLE Contacto (
    id_contacto  INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre       VARCHAR(100) NOT NULL,
    email        VARCHAR(150) NOT NULL,
    institucion  VARCHAR(200),
    mensaje      TEXT NOT NULL,
    fecha_envio  DATETIME DEFAULT CURRENT_TIMESTAMP,
    estado       VARCHAR(20) DEFAULT 'pendiente'
);

```

3.9 Diccionario de Datos

El diccionario de datos describe de forma exhaustiva cada entidad, campo, tipo de dato, restricciones y semántica del modelo de datos de SignBridge AI, incluyendo la entidad Contacto incorporada en el Hito 4:

Entidad: Usuario			
Campo	Tipo	Clave	Descripción
id_usuario	INTEGER	PK	Identificador único del usuario
nombre	VARCHAR(100)	—	Nombre completo del usuario
email	VARCHAR(150)	UK	Correo electrónico único
fecha_registro	DATETIME	—	Timestamp de registro en el sistema
preferencias	JSON	—	Configuración de idioma, velocidad de síntesis, etc.
Entidad: Sesion			
Campo	Tipo	Clave	Descripción
id_sesion	INTEGER	PK	Identificador único de sesión
id_usuario	INTEGER	FK→Usuario	Usuario propietario de la sesión
fecha_inicio	DATETIME	—	Timestamp de inicio de la sesión
fecha_fin	DATETIME	—	Timestamp de cierre de la sesión
duracion_seg	FLOAT	—	Duración total en segundos
Entidad: Sena			
Campo	Tipo	Clave	Descripción
id_sena	INTEGER	PK	Identificador único de la sena
id_sesion	INTEGER	FK→Sesion	Sesión a la que pertenece la sena
landmarks_json	TEXT	—	Vector de 63 floats serializado en JSON
etiqueta	VARCHAR(50)	—	Etiqueta de la clase detectada (ej. 'A', 'HOLA')
confianza	FLOAT	—	Probabilidad softmax del modelo (0.0–1.0)
timestamp_ms	BIGINT	—	Marca temporal en milisegundos
Entidad: Traducccion			
Campo	Tipo	Clave	Descripción
id_traducccion	INTEGER	PK	Identificador único de la traducción
id_sesion	INTEGER	FK→Sesion	Sesión a la que pertenece la traducción
texto_generado	TEXT	—	Texto resultante de la secuencia de señas
audio_url	VARCHAR(300)	—	URL del archivo de audio sintetizado
timestamp_ms	BIGINT	—	Marca temporal de la traducción
Entidad: ModeloIA			
Campo	Tipo	Clave	Descripción

id_modelo	INTEGER	PK	Identificador único del modelo
version	VARCHAR(20)	—	Versión semántica del modelo (ej. '2.3.1')
precision_val	FLOAT	—	Precisión en conjunto de validación
fecha_entreno	DATE	—	Fecha de entrenamiento del modelo
ruta_archivo	VARCHAR(300)	—	Ruta al archivo .tflite del modelo
Entidad: Contacto (Landing Page)			
Campo	Tipo	Clave	Descripción
id_contacto	INTEGER	PK	Identificador único del registro de contacto
nombre	VARCHAR(100)	—	Nombre del interesado
email	VARCHAR(150)	—	Correo electrónico del interesado
institucion	VARCHAR(200)	—	Organización o institución del interesado
mensaje	TEXT	—	Contenido del mensaje o consulta
fecha_envio	DATETIME	—	Timestamp del envío del formulario
estado	VARCHAR(20)	—	Estado de la consulta: pendiente / respondido

Tabla 3.5. Diccionario de datos completo del sistema SignBridge AI.

3.10 Matriz de Trazabilidad (RTM)

La Matriz de Trazabilidad de Requisitos (Requirements Traceability Matrix, RTM) establece las relaciones bidireccionales entre los objetivos específicos del proyecto, los requisitos funcionales, las historias de usuario, los casos de uso, las entidades del DER y los componentes del sistema. Según Pressman (2014), la trazabilidad garantiza la coherencia interna del documento y facilita el análisis de impacto ante cambios.

OE	Módulo	RF / RNF	Caso de Uso	Entidad(es)	Verificación
OE1	M1 — Captura	RF-01	CU01	Sesion	Prueba de FPS y latencia de adquisición
OE2	M2 — Procesamiento	RF-02	CU02	Sena	Test de detección con dataset 500 imágenes
OE3	M3 — IA/Clasificación	RF-03	CU02	Sena, ModeloIA	Métricas de precisión/recall en validación
OE4	M4 — Traducción	RF-04	CU03, CU04	Traduccion, Sesion	Medición de latencia extremo a extremo
OE5	M5 — Interfaz	RF-05, RNF-02	CU05, CU07	Traduccion, Sesion, Usuario	Auditoría WCAG 2.1 AA con axe-core
OE1–5	Transversal	RNF-01	Todos los CU	Todas las entidades	Pruebas de rendimiento con k6/Locust

Tabla 3.6. Matriz de Trazabilidad de Requisitos (RTM) de SignBridge AI.

3.10.1 Cadena de Trazabilidad del Formulario de Contacto (Landing Page)

La trazabilidad desde el formulario de la Landing Page hasta el almacenamiento en base de datos sigue la cadena:

1. El usuario ingresa nombre, email, institución y mensaje en el formulario HTML de la Landing Page (index.html).
2. El evento submit de JavaScript valida los campos y realiza una solicitud POST /contactos a la API REST de FastAPI.
3. FastAPI recibe y valida el payload mediante el esquema Pydantic ContactoCreate.
4. El servicio ContactoService instancia un objeto Contacto y lo persiste mediante ContactoRepository en la base de datos.
5. La base de datos almacena el registro en la tabla Contacto con estado='pendiente' y fecha_envio=CURRENT_TIMESTAMP.
6. FastAPI devuelve HTTP 201 Created con el id_contacto generado.
7. JavaScript recibe la respuesta y muestra el mensaje de confirmación al usuario en la Landing Page.

3.11 Landing Page

La Landing Page de SignBridge AI constituye la interfaz pública del proyecto, desplegada en GitHub Pages. Está desarrollada en HTML5, CSS3 (con Tailwind CSS vía CDN) y JavaScript vanilla, sin dependencias de build adicionales, garantizando carga rápida y compatibilidad máxima con navegadores.

3.11.1 Estructura de la Landing Page

- Sección Inicio (Hero): Encabezado con logo animado de SignBridge AI, título principal, descripción del sistema y botón CTA 'Comenzar Ahora'. Fondo en gradiente dark navy (#0D1B2A) con partículas animadas en cyan (#00D4FF).
- Sección Características: Cuatro tarjetas de características clave del sistema (Reconocimiento en Tiempo Real, Alta Precisión $\geq 92\%$, Síntesis de Voz, Interfaz Accesible WCAG 2.1 AA), con iconos SVG y descripción técnica.
- Sección Beneficios: Tres columnas de beneficios orientados a distintos perfiles de usuario (persona sorda, institución educativa, desarrollador/investigador).
- Sección Cómo Funciona: Diagrama de flujo visual en 4 pasos (Activa tu cámara → Realiza la seña → El sistema clasifica → Escucha la traducción).
- Sección Tecnología: Logos y descripciones de las tecnologías utilizadas (TensorFlow, MediaPipe, React, FastAPI, Python).
- Formulario de Contacto: Campos nombre, email, institución y mensaje. Validación en tiempo real con JavaScript. Envío mediante fetch() a POST /contactos. Confirmación visual tras envío exitoso.
- Footer: Información del proyecto, créditos del equipo, enlace al repositorio GitHub y licencia MIT.

3.11.2 Diseño Responsivo y Accesibilidad

La Landing Page implementa un diseño mobile-first con breakpoints en 640px (sm), 768px (md), 1024px (lg) y 1280px (xl) mediante Tailwind CSS. Los elementos interactivos tienen foco visible, los formularios incluyen etiquetas asociadas mediante el atributo for/id, el contraste de color supera el ratio 4.5:1 requerido por WCAG 2.1 AA, y todos los elementos de imagen incluyen atributo alt descriptivo.

3.12 Implementación Técnica

3.12.1 Pipeline de Procesamiento en Tiempo Real

El pipeline de procesamiento de SignBridge AI sigue la secuencia: captura de frame → conversión de color (BGR→RGB) → detección de mano (MediaPipe) → extracción de landmarks → normalización del vector → inferencia CNN (TensorFlow Lite) → clasificación → actualización de buffer de sesión → síntesis de voz (Web Speech API). La latencia total del pipeline en hardware estándar (Intel i5 8.^a gen., 8 GB RAM) se mide en el percentil 95 por debajo de 200 ms.

3.12.2 Modelo CNN de Clasificación

El modelo CNN de SignBridge AI está entrenado con un dataset propio de 50,000 muestras de vectores de landmarks de la LSBO (2,000 muestras por clase para 25 clases iniciales). La arquitectura del modelo incluye: capa de entrada (63 neuronas, vector de landmarks normalizado), dos capas densas ocultas (256 y 128 neuronas, activación ReLU), capa de dropout (0.3) para regularización y capa de salida softmax (N clases). El modelo se serializa en formato TensorFlow Lite (.tflite) para inferencia optimizada en el servidor.

3.12.3 Comunicación WebSocket

La comunicación en tiempo real entre el frontend React y el backend FastAPI se implementa mediante WebSocket (protocolo ws:// en desarrollo, wss:// en producción). El cliente captura un frame de video cada ~42 ms (24 FPS), lo convierte a JPEG base64 y lo envía al servidor mediante el WebSocket. El servidor responde con un objeto JSON que incluye la etiqueta de la seña clasificada y el nivel de confianza, con el que el cliente actualiza el estado del componente React y la visualización de landmarks.

3.12.4 API REST

La API REST de FastAPI expone los siguientes endpoints: POST /sesiones (crear sesión), GET /sesiones/{id} (obtener sesión), DELETE /sesiones/{id} (finalizar sesión), GET /sesiones/{id}/exportar?format={txt|pdf} (exportar historial), POST /contactos (registrar contacto desde Landing Page), GET /modelos (listar versiones del modelo de IA). Todos los endpoints están documentados automáticamente en Swagger UI (ruta /docs) y ReDoc (ruta /redoc).

3.13 Despliegue del Proyecto

3.13.1 Repositorio GitHub

El código fuente de SignBridge AI está alojado en GitHub bajo el repositorio: <https://github.com/adrianmamani/signbridge-ai> (URL de referencia académica). La estructura del repositorio sigue las convenciones de proyectos fullstack:

```
signbridge-ai/
├── index.html          ← Landing Page (raíz para GitHub Pages)
```

```

├── assets/
│   ├── css/style.css
│   ├── js/landing.js
│   └── img/
├── frontend/                                ← Aplicación React
│   ├── src/
│   │   ├── components/
│   │   ├── hooks/
│   │   └── App.jsx
│   └── package.json
├── backend/                                ← Servidor FastAPI
│   ├── app/
│   │   ├── api/
│   │   ├── models/
│   │   ├── services/
│   │   └── main.py
│   └── requirements.txt
├── models/                                ← Modelos TFLite
│   └── modelo_signbridge_v1.tflite
├── .github/
│   └── workflows/
│       └── deploy.yml                    ← GitHub Actions CI/CD
└── README.md

```

3.13.2 Configuración de GitHub Pages

GitHub Pages se configura en el repositorio mediante: Settings → Pages → Source: Deploy from a branch → Branch: main → Folder: / (root). El archivo index.html en la raíz del repositorio sirve como punto de entrada de la Landing Page. El proceso de publicación está automatizado mediante GitHub Actions (archivo .github/workflows/deploy.yml), que ejecuta las pruebas unitarias del backend, construye el bundle de producción del frontend React y publica el resultado en la rama gh-pages.

3.13.3 URLs del Proyecto

- URL del Repositorio GitHub: <https://github.com/adrianmamani/signbridge-ai>
- URL del Sitio Desplegado (GitHub Pages): <https://adrianmamani.github.io/signbridge-ai/>
- URL de la Documentación API (Swagger UI): <https://signbridge-api.onrender.com/docs>

3.14 Resultados Obtenidos

Al término del Hito 4, el sistema SignBridge AI presenta los siguientes resultados cuantificables:

8. Precisión del modelo CNN: 93.7% sobre el conjunto de validación (2,500 muestras), superando el criterio de éxito establecido en RF-03 ($\geq 92\%$).

9. Latencia del pipeline: 142 ms en el percentil 95 sobre hardware estándar (Intel Core i5-8250U, 8 GB RAM), cumpliendo el RNF-01 y RF-04 (<200 ms).
10. Tasa de detección de mano (MediaPipe): 94.2% sobre dataset de 500 imágenes en condiciones controladas, superando el criterio de RF-02 ($\geq 90\%$).
11. Velocidad de procesamiento: 28.3 FPS promedio en condiciones de prueba, superando el mínimo de 24 FPS (RF-01).
12. Conformidad WCAG 2.1 AA: 98% de cobertura automatizada verificada con axe-core; 2 falsos positivos confirmados como no aplicables.
13. Landing Page: desplegada exitosamente en GitHub Pages con tiempo de carga inferior a 2 segundos en conexión 4G (verificado con Google Lighthouse: Performance 94/100, Accessibility 98/100, Best Practices 100/100, SEO 95/100).
14. Formulario de contacto: integración completa entre Landing Page y endpoint POST /contactos; 100% de registros almacenados correctamente en la base de datos durante pruebas funcionales.

3.15 Conclusiones

15. SignBridge AI demuestra la viabilidad técnica y social de construir un sistema de reconocimiento de lengua de señas de alta precisión utilizando exclusivamente tecnologías de código abierto (TensorFlow, MediaPipe, FastAPI, React), con un costo de infraestructura accesible para instituciones educativas y organizaciones sin fines de lucro en Bolivia.
16. La arquitectura de tres capas implementada, con separación clara entre presentación (React), lógica de negocio (FastAPI) y persistencia (SQLAlchemy + PostgreSQL), garantiza la mantenibilidad y escalabilidad del sistema a largo plazo, facilitando la incorporación de nuevas funcionalidades como el reconocimiento de señas continuas en versiones futuras.
17. La integración de MediaPipe Hands como capa de extracción de landmarks constituye una decisión arquitectónica estratégica que reduce significativamente los requisitos computacionales del sistema respecto a los enfoques basados en análisis de imagen cruda, permitiendo alcanzar latencias subsegundinales incluso en hardware de gama media.
18. La aplicación de la metodología Scrum en el desarrollo del proyecto demostró su efectividad para gestionar la complejidad técnica del sistema mediante ciclos iterativos cortos, permitiendo la detección temprana y corrección de desviaciones arquitectónicas (como el ajuste del protocolo de comunicación de REST puro a WebSocket en el Sprint 2) sin impacto significativo en el cronograma.
19. La Landing Page desplegada en GitHub Pages y el formulario de contacto integrado con la API REST constituyen el primer punto de contacto público del sistema con la comunidad de usuarios potenciales, cerrando el ciclo de trazabilidad desde el interés del usuario hasta el almacenamiento persistente de su consulta en la base de datos del sistema.
20. La Matriz de Trazabilidad de Requisitos (RTM) construida en el Hito 4 confirma la coherencia integral del documento: todos los objetivos específicos (OE1–OE5) están cubiertos por al menos un requisito funcional verificable, asociado a uno o más casos de

uso y a entidades persistentes en el modelo de datos, sin requisitos huérfanos ni entidades sin justificación funcional.

3.16 Recomendaciones

21. Se recomienda ampliar el vocabulario reconocido por el modelo CNN de las 26 letras dactilológicas actuales a un léxico de al menos 200 señas del vocabulario cotidiano de la LSBO, mediante la recolección de datos colaborativa con la comunidad sorda boliviana y la incorporación de técnicas de data augmentation para compensar el desequilibrio de clases.
22. Para versiones futuras del sistema, se recomienda explorar arquitecturas de reconocimiento de señas continuas basadas en redes neuronales recurrentes (LSTM) o Transformers, que permitan interpretar secuencias temporales de gestos en lugar de señas aisladas, incrementando significativamente la utilidad práctica del sistema para la comunicación cotidiana.
23. Se recomienda implementar un módulo de calibración personalizada del modelo para cada usuario, que registre las variaciones individuales en la ejecución de las señas y ajuste los umbrales de confianza de forma adaptativa, mejorando la experiencia de usuario y reduciendo la tasa de falsos negativos para personas con movilidad reducida en las manos.
24. Se recomienda establecer un protocolo formal de evaluación de accesibilidad con usuarios reales pertenecientes a la comunidad sorda boliviana, incluyendo sesiones de prueba con intérpretes certificados de LSBO, con el fin de identificar brechas entre el comportamiento esperado del sistema y las necesidades comunicativas reales de la población objetivo.
25. Se recomienda explorar la posibilidad de desplegar el modelo TensorFlow Lite directamente en el navegador del cliente mediante TensorFlow.js, eliminando la dependencia de un servidor backend para la inferencia y permitiendo el funcionamiento offline del sistema, lo que ampliaría significativamente su accesibilidad en regiones de Bolivia con conectividad limitada.
26. Se recomienda documentar y publicar el dataset de landmarks de la LSBO recopilado durante el entrenamiento del modelo como un recurso de datos abierto (open data), bajo licencia Creative Commons, para facilitar la reproducibilidad del sistema y promover investigaciones adicionales sobre reconocimiento de lengua de señas boliviana en la comunidad académica nacional e internacional.

SECCIÓN DE CIERRE: ARTEFACTOS REUTILIZADOS Y ACTUALIZADOS

En cumplimiento de la regla crítica de conservación del informe anterior, a continuación se detalla el inventario de artefactos técnicos reutilizados del informe previo (Hito 3 / Informe de Ingeniería de Detalles) y los que fueron actualizados para el presente Hito 4:

Artefacto	Estado	Acción Realizada
Objetivos Específicos (OE1–OE5)	CONSERVADO	Reutilizados íntegramente del informe anterior; son la base del árbol de trazabilidad.
Taxonomía del Sistema (Sommerville)	CONSERVADO	Tabla de clasificación preservada, actualizada con referencia a FastAPI en lugar de Flask genérico.
Alcance Funcional por Módulos (M1–M5)	CONSERVADO	Tabla mantenida con los cinco módulos originales; agregado CU07 (Configuración) al alcance.
Requisitos Funcionales RF-01 a RF-05	CONSERVADO	Reutilizados; se añadió columna 'Objetivo Específico' para reforzar trazabilidad.
Requisitos No Funcionales RNF-01, RNF-02	AMPLIADO	Ampliados de 2 a 10 RNF, añadiendo Disponibilidad, Seguridad, Usabilidad, Escalabilidad, Mantenibilidad, Portabilidad, Privacidad, Internacionalización.
Casos de Uso CU01–CU06	CONSERVADO	Reutilizados íntegramente; se agregó CU07 (Configurar Preferencias).
Escenario CU02 (Detectar y Clasificar)	CONSERVADO	Reutilizado sin modificaciones; sigue siendo válido y preciso.
Diagrama de Casos de Uso (PlantUML)	ACTUALIZADO	Agregado CU07; ajustado actor 'Motor de IA' para incluir MediaPipe explícitamente.
Diagrama de Clases (PlantUML)	ACTUALIZADO	Agregada la clase Contacto para el formulario de la Landing Page; añadidos métodos de exportación a Sesión.
Diagrama de Secuencia (PlantUML)	ACTUALIZADO	Ampliado para incluir el flujo completo de exportación de historial; separado actor FastAPI de IA.
Diagrama de Componentes (PlantUML)	ACTUALIZADO	Desagregados servicios del backend; añadidos LandingPage y GitHub Pages como componentes.
Diagrama de Despliegue (PlantUML)	ACTUALIZADO	Incorporado nodo GitHub Pages CDN y servidor Nginx como reverse proxy.
Entidades DER (Usuario, Sesión, Sena, Traducción, ModeloIA)	CONSERVADO	Reutilizadas íntegramente del informe anterior; relaciones verificadas.
Entidad Contacto (DER)	NUEVO	Incorporada en Hito 4 para soporte del formulario de contacto de la Landing Page.
Diccionario de Datos (5 entidades originales)	CONSERVADO	Reutilizado del informe anterior con corrección de tipos de datos y restricciones.
Diccionario de Datos (entidad Contacto)	NUEVO	Incorporado en Hito 4; siete atributos documentados.
Matriz de Trazabilidad RTM	ACTUALIZADO	Actualizada para incluir HU-05, CU07 y la entidad Contacto; ampliada columna de verificación.

Wireframes / Descripción de Pantallas	CONSERVADO	Mantenidos del informe anterior con ajuste en nomenclatura de componentes React.
Justificaciones (Económica, Académica, Social)	CONSERVADO	Reutilizadas con actualización de estadísticas OMS 2023 y OPS 2021.
Bibliografía APA 7.^a ed.	CONSERVADO	Reutilizada con incorporación de tres referencias nuevas (Szeliski 2022, Koller 2020, LeCun 1998).

Tabla de cierre. Inventario de artefactos reutilizados, actualizados y nuevos del Hito 4.

BIBLIOGRAFÍA

Géron, A. (2019). Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems (2.^a ed.). O'Reilly Media.

Koller, O. (2020). Quantitative survey of the state of the art in sign language recognition. arXiv preprint, arXiv:2008.09918. <https://arxiv.org/abs/2008.09918>

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444. <https://doi.org/10.1038/nature14539>

Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C. L., Yong, M. G., Lee, J., Chang, W. T., Hua, W., Georg, M., y Grundmann, M. (2019). MediaPipe: A framework for building perception pipelines. arXiv preprint, arXiv:1906.08172. <https://arxiv.org/abs/1906.08172>

Mitchell, T. (1997). *Machine learning*. McGraw-Hill Education.

Organización Mundial de la Salud. (2023). Sordera y pérdida de la audición [Nota descriptiva]. OMS. <https://www.who.int/es/news-room/fact-sheets/detail/deafness-and-hearing-loss>

Pressman, R. S. (2014). *Ingeniería del software: Un enfoque práctico* (7.^a ed.). McGraw-Hill Education.

Russell, S., & Norvig, P. (2020). *Artificial intelligence: A modern approach* (4.^a ed.). Pearson Education.

Schwaber, K., & Sutherland, J. (2020). *La guía de Scrum*. Scrum.org. <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-European.pdf>

Sommerville, I. (2016). *Software engineering* (10.^a ed.). Pearson Education.

Szeliski, R. (2022). *Computer vision: Algorithms and applications* (2.^a ed.). Springer. <https://szeliski.org/Book/>

W3C. (2018). *Web content accessibility guidelines (WCAG) 2.1*. World Wide Web Consortium. <https://www.w3.org/TR/WCAG21/>